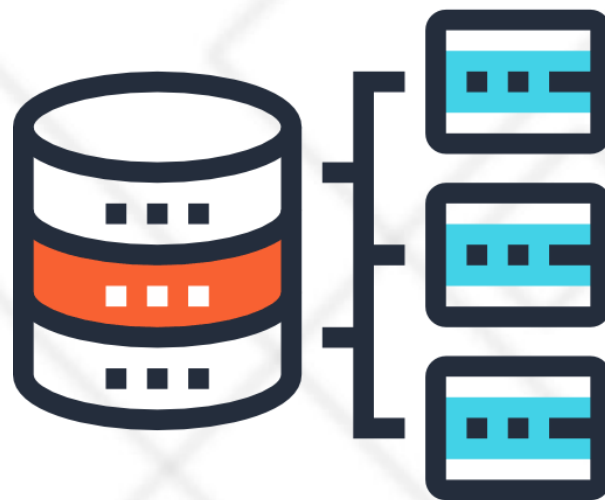




北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY



第7章 数据库的完整性

北京理工大学 计算机学院

段春晖

duanch@bit.edu.cn

第7章 数据库的完整性

□数据库的完整性

- 数据的**正确性**和**相容性**

- ◆ 防止数据库中存在不符合语义的数据，也就是防止数据库中存在不正确的数据
- ◆ 防范对象：不合语义的、不正确的数据

□数据的安全性

- 保护数据库防止恶意的破坏和非法的存取
- 防范对象：非法用户和非法操作



第7章 数据库的完整性

- 7.1 数据库的完整性概述
- 7.2 实体完整性
- 7.3 参照完整性
- 7.4 用户定义的完整性
- 7.5 触发器
- 7.6 SQL Server中数据库完整性的实现
- 7.7 小结

7.1 数据库的完整性概述

- 数据库的完整性是一种语义概念
 - 防止数据库中存在
 - ◆ 不符合语义的数据
 - ◆ 不正确的数据
 - 保证数据库中数据的质量
- 为维护数据库的完整性，DBMS必须：
 - 提供定义完整性约束条件的机制
 - 提供完整性检查的方法
 - 违约处理

7.1 数据库的完整性概述

□ 7.1.1 完整性约束条件

- 在关系数据库中，对数据的各种限制是以完整性约束条件的形式在关系数据库模式中指定的
 - ◆ 加在数据上的语义约束条件称为数据库完整性约束条件
 - ◆ 它一般是对数据库中数据本身的某种语义限制、数据间的逻辑约束和数据变化时所遵循的规则等

7.1.1 完整性约束条件

□(1) 列级约束

- 主要是对属性的数据类型、数据格式和取值范围、精度等的约束。具体包括：
 - ◆ 对数据类型的约束，包括数据类型、长度、精度等的约束。例如学生姓名的数据类型是字符型，长度是8
 - ◆ 对数据格式的约束，例如规定日期的格式为YYYY/MM/DD
 - ◆ 对取值域的约束，例如学生成绩的取值范围必须是0 ~ 100
 - ◆ 对空值的约束

7.1.1 完整性约束条件

□(2) 元组约束

- 一个元组是由若干个属性组成的，元组级约束就是元组中各个属性之间的约束关系。例如订货关系中发货日期不能小于订货日期，发货量不得超过订货量等

□(3) 关系约束

- 关系约束是指一个关系的各个元组之间、或者多个关系之间存在的各种联系或约束。常见的关系约束有实体完整性约束、参照完整性约束、函数依赖约束、统计约束等

7.1.2 实现数据完整性的方法

□在关系数据库系统中，数据完整性控制策略包括规则、默认值、约束、触发器和存储过程等

□(1) 默认值

- 如果在插入行中没有指定列的值，那么默认值指定列中所使用的值，例如：自动增长值，内置函数、数学表达式等

□(2) 约束

- 约束是自动强制数据完整性的方法
- 约束定义关系列中允许值的规则，是通用的强制完整性的标准机制
- 使用约束优于使用触发器、规则和默认值

7.1.2 实现数据完整性的方法

□(3) 规则

- 规则是大多数数据库系统中一个向后兼容的功能，用于执行一些与CHECK约束相同的功能。规则以单独的对象创建，然后绑定到列上

□(4) 触发器

- 触发器是数据库系统中强制业务规则和数据完整性的主要机制



第7章 数据库的完整性

- 7.1 数据库的完整性概述
- 7.2 实体完整性
- 7.3 参照完整性
- 7.4 用户定义的完整性
- 7.5 触发器
- 7.6 SQL Server中数据库完整性的实现
- 7.7 小结

7.2 实体完整性

□ 实体完整性规则规定：

- 主键的值不能取空值
- 主键的值唯一

□ 实现方法：通过对主键值的约束实现实体完整性

□ 7.2.1 实体完整性的定义

- 关系模型的实体完整性
 - ◆ CREATE TABLE中用PRIMARY KEY定义
- 单属性构成的主键有两种说明方法
 - ◆ 定义为列级约束条件、定义为表级约束条件
- 对多个属性构成的主键只有一种说明方法
 - ◆ 定义为表级约束条件



7.2.1 实体完整性的定义

- PRIMARY KEY约束可以作为表定义的一部分在创建表时定义，也可以在表创建之后再添加

```
CREATE TABLE Student
(Sno CHAR(9) PRIMARY
KEY,
Sname CHAR(20) NOT
NULL,
Ssex CHAR(2),
Sage SMALLINT,
Sdept CHAR(20));
```

```
CREATE TABLE Student
(Sno CHAR(9),
Sname CHAR(20) NOT
NULL,
Ssex CHAR(2),
Sage SMALLINT,
Sdept CHAR(20),
PRIMARY KEY (Sno));
```

7.2.1 实体完整性的定义

- 为了实施实体完整性，一般地，系统将在主键属性上自动创建唯一的索引来强制唯一性约束
- **[例]** 将SC表中的Sno, Cno属性组定义为码

CREATE TABLE SC

(Sno CHAR(9) NOT NULL,

Cno CHAR(4) NOT NULL,

Grade SMALLINT,

PRIMARY KEY (Sno, Cno) /*只能在表级定义主键*/

);

7.2.2 实体完整性检查和违约处理



□使用全表扫描法检查记录中主键值是否唯一

待插入记录

Key _i	F2 _i	F3 _i	F4 _i	F5 _i
------------------	-----------------	-----------------	-----------------	-----------------

基本表

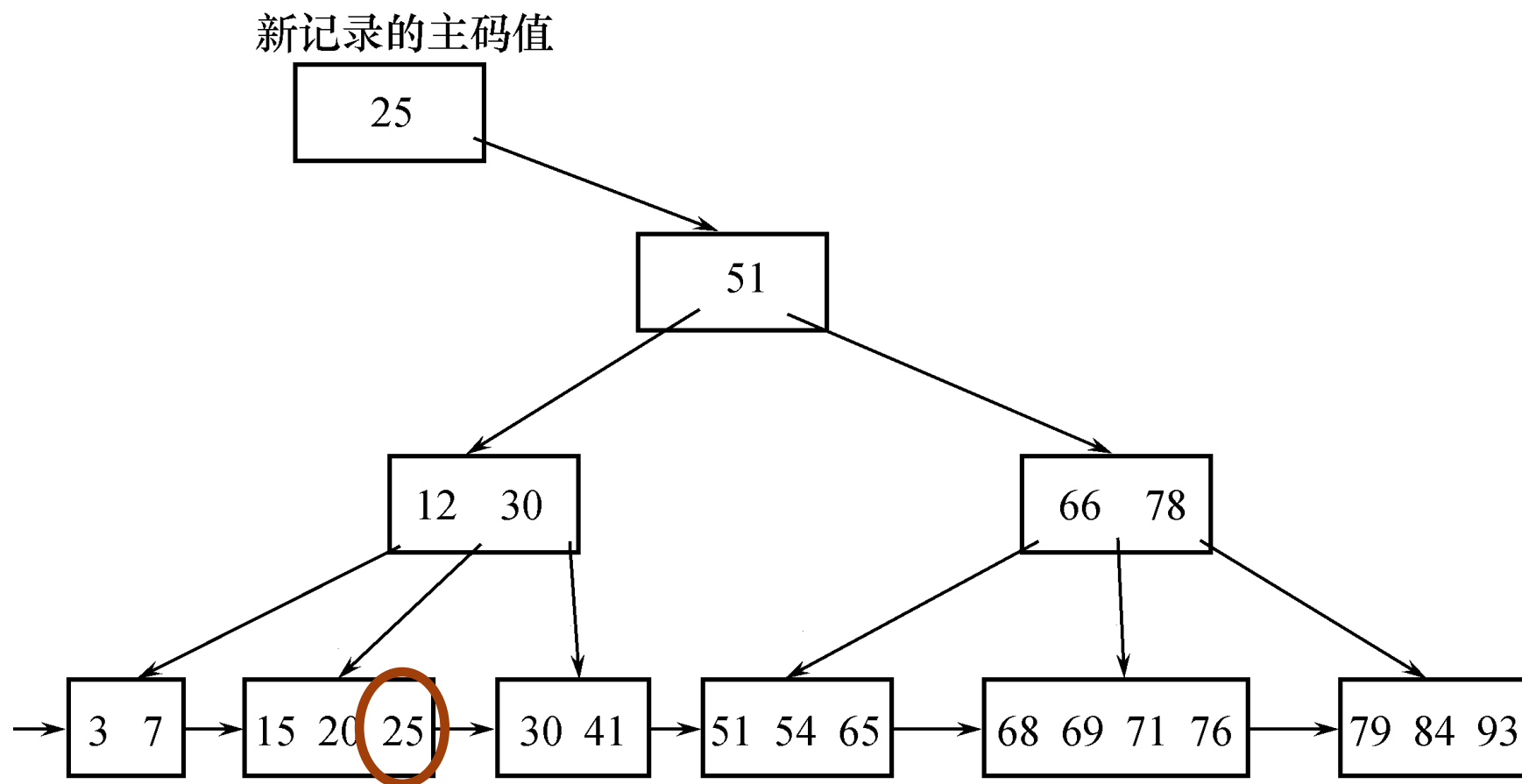
Key1	F21	F31	F41	F51
Key2	F22	F32	F42	F52
Key3	F23	F33	F43	F53
⋮				



7.2.2 实体完整性检查和违约处理



□使用索引判断主键是否存在





7.2.2 实体完整性检查和违约处理



- 定义表的主键后，每当对该表插入一条记录或者对主键进行更新操作时，DBMS自动进行实体完整性的检查
 - 检查主键是否唯一
 - ◆ 如果不唯一则拒绝进行插入或修改
 - 检查主键的各个属性（字段）值是否为空
 - ◆ 如果有空的字段值，则拒绝操作，从而保证实体完整性

7.3 参照完整性

□ 7.3.1 参照完整性定义

- 关系模型的参照完整性定义
 - ◆ 用FOREIGN KEY短语定义哪些列为外键
 - ◆ 用REFERENCES短语指明外键参照哪些表的主键

□ [例 7-6] 定义SC中的参照完整性

CREATE TABLE SC

(Sno CHAR(9) NOT NULL,

Cno CHAR(4) NOT NULL,

Grade SMALLINT,

PRIMARY KEY (Sno, Cno),

FOREIGN KEY (Sno) REFERENCES Student(Sno),

FOREIGN KEY (Cno) REFERENCES Course(Cno));

主表与从表

□ **主表**：指外键在另一张表中作主/候选键的表
(例中的班级表)

□ **从表**：指含有外键的表 (例中的学生表)

学生表 (从表)

姓名	<u>学号</u>	D.O.B	班号
Peter	2020-101	5-10-1996	计算机202001

班级表 (主表)

<u>班号</u>	班名	人数	...
计算机202001	计1班	50	...
计算机202010	计10班	45	...

外键约束的例子 (主表→从表)

学生表 (从表)

姓名	<u>学号</u>	D.O.B	班号
Peter	2020-101	5-10-1996	计算机 202001

外键

主键

班级表 (主表)

<u>班号</u>	班名	人数	...
计算机 202001	计1班	50	...
计算机 202010	计10班	45	...

删除

修改

插入

分析外键约束 (主表→从表)

- 对主表中的主键进行三种操作，观察对表间完整性的影响
 - **插入**：要求插入值满足主键限制即可，不影响其它表
 - **修改**：可能会影响与该主键相关的从表的外键值。当相应的外键值存在时，有两个策略可用：一是**改变对应从表的所有外键值**，使之与主键一致；其二是**不允许修改主表中的主键值**
 - **删除**：可能会影响与该主键相关的从表的外键值。若相应的外键值存在时，策略有二：一是**不允许删除主表的主键值**；二是**级联删除**从表中相应外键值所在的行

外键约束的例子 (从表→主表)

学生表 (从表)

姓名	<u>学号</u>	D.O.B	班号
Peter	2020-101	5-10-1996	计算机202001

外键

插入

删除

主键

修改

班级表 (主表)

<u>班号</u>	班名	人数	...
计算机202001	计1班	50	...
计算机202010	计10班	45	...

分析外键约束 (从表→主表)

□对从表中的外键操作

- **插入**：要求插入的外键值应参照 (Reference) 主表中的主键值
- **修改**：要求修改的外键值参照主表中的主键值
- **删除**：不需要参照主表中的主键值



7.3.2 参照完整性检查和违约处理



可能破坏参照完整性的情况及违约处理

被参照表（例如班级表）	参照表（例如学生表）	违约处理
可能破坏参照完整性 ←	插入元组	拒绝
可能破坏参照完整性 ←	修改外键值	拒绝
删除元组	→ 可能破坏参照完整性	拒绝/级联删除 /设置为空值
修改主键值	→ 可能破坏参照完整性	拒绝/级联修改 /设置为空值

7.3.2 参照完整性检查和违约处理

□ 1. 参照关系中外键空值的问题

- 需要定义外键是否允许为空值
- 如果外键是其主键的组成部分，外键值不允许为空
- 否则可以根据具体的语义确定外键值是否允许空值

姓名	<u>学号</u>	D.O.B	班号
Peter	2020-101	5-10-1996	计算机202001

学生表（从表）

班级表（主表）

<u>班号</u>	班名	人数	...
计算机202001	计1班	50	...
计算机202010	计10班	45	...



7.3.2 参照完整性检查和违约处理



- 2. 在参照关系中插入元组的问题（修改操作与之类似）
 - 受限插入
 - ◆ 向学生表中插入新的元组，但该元组的班号属性值在班级表中不存在，则系统拒绝
 - 级联插入 (CASCADE)
 - ◆ 首先向被参照关系插入相应的元组，其主键值等于参照关系插入元组的外键值，然后再向参照关系插入该元组



7.3.2 参照完整性检查和违约处理



□ 3. 在被参照关系中删除元组的问题

- 级联删除 (CASCADE)

- ◆ 将参照关系中所有外键值与被参照关系中要删除元组主键值相同的元组一起删除
- ◆ 如果参照关系同时又是另一个关系的被参照关系, 这种删除操作会继续级联下去



7.3.2 参照完整性检查和违约处理



□ 3. 在被参照关系中删除元组的问题

- 级联删除 (CASCADE)
- 受限删除 (RESTRICTED)
 - ◆ 仅当参照关系中没有任何元组的外键值与被参照关系中要删除元组的主键值相同时，系统才执行删除操作，否则拒绝删除操作
 - ◆ 例如果学生表中存在“班号 = 202001”的元组，则系统拒绝删除班级表中“班号 = 202001”的元组

7.3.2 参照完整性检查和违约处理

□ 3. 在被参照关系中删除元组的问题

- 置空值删除

- ◆ 删除被参照关系的元组，并将参照关系中相应元组的外键值置为空值

姓名	<u>学号</u>	D.O.B	班号
Peter	2020-101	5-10-1996	计算机202001

学生表（从表）

<u>班号</u>	班名	人数	...
计算机202001	计1班	50	...
计算机202010	计10班	45	...

班级表（主表）



7.3.2 参照完整性检查和违约处理



- 3. 在被参照关系中删除元组的问题
 - 级联删除 (CASCADE)
 - 受限删除 (RESTRICTED)
 - 置空值删除
- 这三种方法都保证了参照完整性约束，但具体哪种方法正确，取决于应用环境的语义，需要根据实际应用的业务规则决定具体的违约处理措施
- 如果想让系统采用其他的策略则必须在创建表时显式地加以说明

□显式说明参照完整性的违约处理示例

```
CREATE TABLE SC
```

```
(Sno CHAR(9) NOT NULL,
```

```
Cno CHAR(4) NOT NULL,
```

```
Grade SMALLINT,
```

```
PRIMARY KEY (Sno, Cno),
```

```
FOREIGN KEY (Sno) REFERENCES Student(Sno)
```

```
    ON DELETE CASCADE /*级联删除SC表中相应的元组*/
```

```
    ON UPDATE CASCADE, /*级联更新SC表中相应的元组*/
```

```
FOREIGN KEY (Cno) REFERENCES Course(Cno)
```

```
    ON DELETE NO ACTION
```

```
/*当删除Course表中的元组造成了与SC表不一致时拒绝删除*/
```

```
    ON UPDATE CASCADE
```

```
/*当更新Course表中的Cno时, 级联更新SC表中相应的元组*/
```

```
);
```

7.4 用户定义的完整性

□ 用户定义的完整性就是针对**某一具体应用**的数据必须满足的语义要求

- RDBMS提供，而不必由应用程序承担

□ 7.4.1 用户定义完整性定义

- 属性上的约束条件的定义

- ◆ 通常属性上的约束条件包括：唯一 (UNIQUE)、非空 (NOT NULL)、CHECK约束、默认值DEFAULT等

- 元组上的约束条件的定义

- ◆ 用CHECK短语定义元组上的约束条件，即元组级的限制，设置不同属性之间的相互约束条件

7.4.1 用户定义完整性定义

- 元组级的限制可以设置不同属性之间的取值的相互约束条件
- [例]** 当学生的性别是男时，其名字不能以Ms.打头

CREATE TABLE Student

(Sno CHAR(9),

Sname CHAR(8) NOT NULL,

Ssex CHAR(2), Sage SMALLINT,

Sdept CHAR(20), PRIMARY KEY (Sno),

CHECK (Ssex='女' OR Sname NOT LIKE 'Ms.%')

/*定义了元组中Sname和Ssex两个属性值之间的约束条件*/);

- 性别是女性的元组都能通过检查，因Ssex='女'成立
- 当性别是男性时，要通过检查则名字不能以Ms.打头

7.4.1 用户定义完整性定义

□[例 7-8] 创建table1，指定c1字段不能包含重复值，c2字段只能取特定值

```
CREATE TABLE table1
```

```
(c1 CHAR(2) UNIQUE,
```

```
c2 CHAR(4) CHECK (c2 IN ('0000', '0001', '0002',  
'0003')),
```

```
c3 INT DEFAULT 1
```

```
);
```

□违约处理

- 插入元组或修改属性的值时，RDBMS检查元组上的约束条件是否被满足，如果不满足则拒绝执行

完整性约束命名子句

□ 定义CONSTRAINT约束

CONSTRAINT <完整性约束条件名>

[PRIMARY KEY短语

| FOREIGN KEY短语

| CHECK短语];

完整性约束命名子句

- **[例]** 建立学生登记表Student, 要求学号在90000~99999之间, 姓名不能取空值, 年龄小于30, 性别只能是“男”或“女”

CREATE TABLE Student

(Sno NUMERIC(6)

CONSTRAINT C1 CHECK (Sno BETWEEN 90000 AND 99999),

Sname CHAR(20)

CONSTRAINT C2 NOT NULL,

Sage NUMERIC(3)

CONSTRAINT C3 CHECK (Sage < 30),

Ssex CHAR(2)

CONSTRAINT C4 CHECK (Ssex IN ('男', '女')),

CONSTRAINT StudentKey PRIMARY KEY (Sno));

- 在Student表上建立了5个约束条件, 包括主键约束 (命名为StudentKey) 以及C1、C2、C3、C4四个列级约束

完整性约束命名子句

□修改表中的完整性限制

- 使用ALTER TABLE语句修改表中的完整性限制

□**[例]** 修改表Student中的约束条件，要求学号改为在900000~999999之间，年龄小于30改为小于40
可以先删除原来的约束条件，再增加新的约束条件

ALTER TABLE Student

DROP CONSTRAINT C1;

ALTER TABLE Student

**ADD CONSTRAINT C1 CHECK (Sno BETWEEN
900000 AND 999999),**

ALTER TABLE Student

DROP CONSTRAINT C3;

ALTER TABLE Student

ADD CONSTRAINT C3 CHECK (Sage < 40);



第7章 数据库的完整性

- 7.1 数据库的完整性概述
- 7.2 实体完整性
- 7.3 参照完整性
- 7.4 用户定义的完整性
- 7.5 触发器
- 7.6 SQL Server中数据库完整性的实现
- 7.7 小结

7.5 触发器

□数据库系统一般提供两种主要机制来实现业务规则和数据完整性

- 约束

- ◆完整性约束机制在检测出违反约束条件的操作后，只能做简单的动作，例如，拒绝操作

- 触发器

- ◆触发器是用户定义在关系数据表上的一类由事件驱动的特殊过程，用编程的方法实现复杂的业务规则

- ◆触发器比约束更加灵活，可以实现一般的数据完整性约束实现不了的复杂的完整性约束，具有更精细和更强大的数据控制能力

- ◆触发器常常用于强制业务规则和数据完整性

7.5.1 触发器概述

- 触发器是一种特殊类型的存储过程，在对表或视图发出UPDATE、INSERT或DELETE语句时**自动执行**
- 可用触发器完成很多数据库完整性保护的功能
 - 实现复杂的业务规则
 - 实现比CHECK约束更复杂的数据完整性
 - 比较数据修改前后的状态
 - 维护非规范化数据

7.5.2 定义触发器

- SQL使用CREATE TRIGGER命令创建触发器，其一般格式为

CREATE TRIGGER <触发器名>

{BEFORE | AFTER} <触发事件> ON <表名>

FOR EACH {ROW | STATEMENT}

[WHEN <触发条件>]

<触发动作体>;

- 定义触发器的语法说明：

1. 创建者：表的**拥有者**
2. 触发器名，表名：触发器的目标表
3. 触发事件：INSERT、DELETE、UPDATE

7.5.2 定义触发器

4. 触发时间

- BEFORE

- ◆ 表示在触发事件进行以前，判断触发条件是否满足
- ◆ 若满足条件则先执行触发动作部分的操作
- ◆ 然后再执行触发事件的操作

- AFTER

- ◆ 表示在触发事件完成之后，判断触发条件是否满足
- ◆ 若满足条件则执行触发动作部分的操作
- ◆ 如果触发事件因错误（如违反约束或语法错误）而失败，触发器将不会执行

7.5.2 定义触发器

5. 触发器类型

- 行级触发器 (FOR EACH ROW)
 - ◆ 对每一个修改的元组都会触发触发器的检查和执行
 - 语句级触发器 (FOR EACH STATEMENT)
 - ◆ 只在SQL语句执行时候进行触发条件的检查和触发器的执行
- 例如Teacher表上创建一个AFTER UPDATE触发器。如果表Teacher有1000行，执行如下语句：
- UPDATE Teacher SET Deptno=5;**
- 如果该触发器为语句级触发器，那么执行完该语句后，触发动作只发生一次
 - 如果是行级触发器，触发动作将执行1000次

7.5.2 定义触发器

6. 触发条件

- 触发器被激活时，只有当触发条件为真时触发动作体才执行；否则触发动作体不执行
- 如果省略WHEN触发条件，则触发动作体在触发器激活后立即执行

7. 触发动作体

- 触发动作体是满足触发器条件后，执行的一系列数据库操作
- 如果触发动作体执行失败，激活触发器的事件就会终止执行，触发器的目标表或触发器可能影响的其他对象不发生任何变化

注意：不同的RDBMS产品触发器语法各不相同！

7.5.3 使用触发器

□通过触发器实现数据完整性

□**[例]** 定义一个BEFORE行级触发器，为教师表Teacher定义完整性规则 “教授的工资不得低于4000元，如果低于4000元，自动改为4000元”

```
CREATE TRIGGER Insert_Or_Update_Sal
```

```
BEFORE INSERT OR UPDATE ON Teacher
```

```
/*触发事件是插入或更新操作*/
```

```
FOR EACH ROW /*行级触发器*/
```

```
AS BEGIN /*定义触发动作体，是PL/SQL过程块*/
```

```
IF (new.Job='教授') AND (new.Sal < 4000)
```

```
THEN new.Sal := 4000;
```

```
END IF;
```

```
END;
```

7.5.3 使用触发器

□[例] 定义AFTER行级触发器，当教师表Teacher的工资发生变化后就自动在工资变化表Sal_log中增加一条相应记录

- 首先建立工资变化表Sal_log

```
CREATE TABLE Sal_log
```

```
(Eno NUMERIC(4) REFERENCES Teacher(Eno),
```

```
Sal NUMERIC(7, 2),
```

```
Username CHAR(10),
```

```
Date TIMESTAMP
```

```
);
```

7.5.3 使用触发器

□[例] 定义AFTER行级触发器，当教师表Teacher的工资发生变化后就自动在工资变化表Sal_log中增加一条相应记录

● (续)

```
CREATE TRIGGER Insert_Sal
```

```
AFTER INSERT ON Teacher /*触发事件是INSERT*/
```

```
FOR EACH ROW
```

```
AS BEGIN
```

```
    INSERT INTO Sal_log VALUES (new.Eno, new.Sal,  
                                CURRENT_USER, CURRENT_TIMESTAMP);
```

```
END;
```

7.5.3 使用触发器

□[例] 定义AFTER行级触发器，当教师表Teacher的工资发生变化后就自动在工资变化表Sal_log中增加一条相应记录

● (续)

```
CREATE TRIGGER Update_Sal
  AFTER UPDATE ON Teacher /*触发事件是UPDATE*/
  FOR EACH ROW
  AS BEGIN
    IF (new.Sal <> old.Sal) THEN INSERT INTO Sal_log
      VALUES (new.Eno, new.Sal, CURRENT_USER,
        CURRENT_TIMESTAMP);
    END IF;
  END;
```

7.5.3 使用触发器

- 触发器的执行，是由触发事件激活的，并由数据库服务器自动执行
- 一个数据表上可能定义了多个触发器
 - 同一个表上的多个触发器激活时遵循如下的执行顺序
 - (1) 执行该表上的BEFORE触发器
 - (2) 激活触发器的SQL语句
 - (3) 执行该表上的AFTER触发器

7.5.3 使用触发器

□ 执行修改某个教师工资的SQL语句，激活上述定义的触发器

UPDATE Teacher SET Sal=800 WHERE Ename=‘陈平’;

● 执行顺序是：

- ◆ 执行触发器Insert_Or_Update_Sal
- ◆ 执行SQL语句 “UPDATE Teacher SET Sal=800 WHERE Ename=‘陈平’;”
- ◆ 执行触发器Insert_Sal
- ◆ 执行触发器Update_Sal

删除触发器

□ 删除触发器的SQL语法:

DROP TRIGGER <触发器名> ON <表名>;

□ 触发器必须是一个已经创建的触发器，并且只能由具有相应权限的用户删除

□ **[例]** 删除教师表Teacher上的触发器Insert_Sal

DROP TRIGGER Insert_Sal ON Teacher;

决定使用何种强制方法

- 应综合考虑功能性和性能开销，来决定使用何种强制数据完整性的方法
 - 对于基本的完整性逻辑，例如有效值和维护表间的关系，最好使用声明式完整性约束
 - 如果要维护复杂的、大量的、非主键或外键关系一部分的数据，必须使用触发器或存储过程



第7章 数据库的完整性

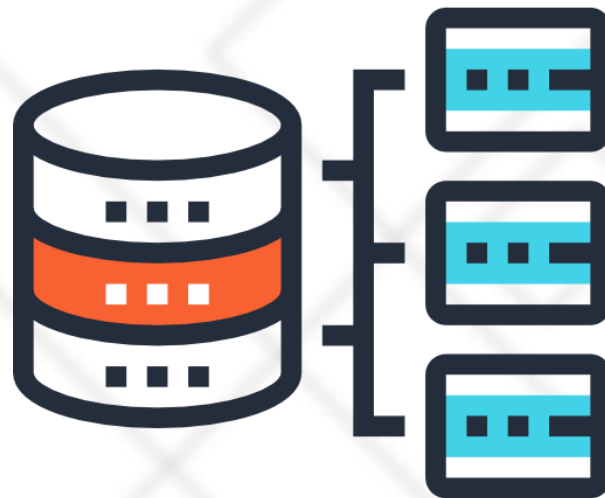
- 7.1 数据库的完整性概述
- 7.2 实体完整性
- 7.3 参照完整性
- 7.4 用户定义的完整性
- 7.5 触发器
- 7.6 SQL Server中数据库完整性的实现
- 7.7 小结

7.7 小结

- 数据库的完整性是为了保证数据库中存储的数据是正确的
- RDBMS完整性实现的机制
 - 完整性约束定义机制
 - 完整性检查机制
 - 违背完整性约束条件时RDBMS应采取的动作
- 实体完整性
- 参照完整性
- 用户定义的完整性
- 实现数据库完整性的一个重要方法——触发器



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY



作业

《数据库系统原理》第7章

习题10, 12

□实验3：复杂查询和触发器

□实验目的

- 掌握SQL嵌套查询和集合查询等各种高级查询的设计方法；掌握数据库触发器的设计和使用方法

□实验内容

- 详见乐学课程

□提交方式

- 打包提交：实验报告、源文件（.sql文件）
- 压缩文件命名方式：学号-姓名-实验X